Synthetic Data Compilation for Rollout-Free Trajectory Planning

Thomas Y.L. Lin [1]

Abstract

We revisit model-based reinforcement learning and remove recurrent rollout for downstream planning. Current action-conditioned models confront an efficiency-accuracy tradeoff for prediction: learning immediate next-state transitions requires fine temporal discretization and numerous rollout steps, while accurate regression needs massive trajectory data. We solve this tension by distilling a teacher representation of instantaneous dynamics into a student predictor. This gives both planning and training advantages. For planning, the student maps an action sequence to its consequences, avoiding recurrent rollout and backpropagation through a recurrent graph. For training, the teacher compiles synthetic supervision from any well-posed integration of the learned dynamics, improving generalization across trajectory distributions while preserving consistency with the governing law. We provide theoretical justification and empirical evidence for these advantages.

1 Introduction

We revisit model-based reinforcement learning (MBRL) and remove recurrent rollout for downstream planning. Our motivation comes from mainstream action-conditioned world models (Hafner et al., 2020; Hansen et al., 2022b) that map a current state and action to the immediate next state, implicitly assuming locally constant actions under zero-order hold (ZOH). Such modeling relies on fine temporal discretization and thus requires numerous recurrent rollout steps for planning. For example, with $\Delta t \approx 0.02$s, a 10-second query requires 500 rollout steps (Tassa et al., 2018; Hafner et al., 2020; Hansen et al., 2022b). Moreover, action planning through this recurrent graph becomes costly and susceptible to vanishing or exploding gradients (Pascanu et al., 2013).

Removing recurrent rollout suggests learning an endpoint predictor that maps an initial state and an action sequence directly to its consequence. However, direct endpoint regression is high-dimensional, so trajectory data coverage becomes sparse as the action-sequence length grows. Together, rollout-based modeling and endpoint prediction expose a tradeoff between planning cost and large-scale supervision.

We resolve this tradeoff by compiling synthetic data from a learned teacher dynamics model. The teacher predicts the instantaneous state derivative from local state-action pairs, i.e. $\dot{z} = f(z, u)$. For each supported action sequence, we integrate the teacher along the induced trajectory and use the resulting endpoint to distill a student predictor.

Advantages. The proposed method (Figure 1) shows three advantages:

- **Remove Planning Rollout.** The endpoint predictor replaces recurrent rollout with direct consequence query. When the reward objective uses K checkpoints (9), the model evaluates K endpoint queries that can be batched in parallel rather than sequentially (Section 3).
- **Direct Action Gradients.** Planning through the predictor optimizes the action sequence by backpropagating through a single network instead of a recurrent one. This removes products of Jacobians and gradient sensitivity in rollout planning (Lemma 3.1 and Corollary 3.2).
- **Synthetic Endpoint Supervision.** We compile synthetic endpoint targets by integrating the teacher along supported induced trajectories. This improves coverage across high-dimensional trajectory distributions while preserving consistency with the governing law (Section 4 and Proposition 4.1).

Roadmap. Section 2 formalizes. Sections 3 and 4 justify the advantages. Sections B and 5 report empirical tests. Section 6 concludes. Section A reviews related work.

2 Endpoint World Model

Controlled dynamical law. For latent state $z(t) \in \mathbb{R}^d$ and control $u(t) \in \mathbb{R}^m$, assume the governing law f follows

$$\frac{\mathrm{d}z}{\mathrm{d}t} = f(z(t), u(t)).$$

Note this law is autonomous: all relevant exogenous variables are included in z or u, so f has no explicit time dependency. For any well-posed control signal $u_{[t,t+T]} := u(\cdot)|_{[t,t+T]}$, f induces an *endpoint operator* Φ_T^f mapping

[1]University of Washington, Seattle, USA. Correspondence to: Thomas Y.L. Lin <thermaus@uw.edu>.

Preprint. June 11, 2026.

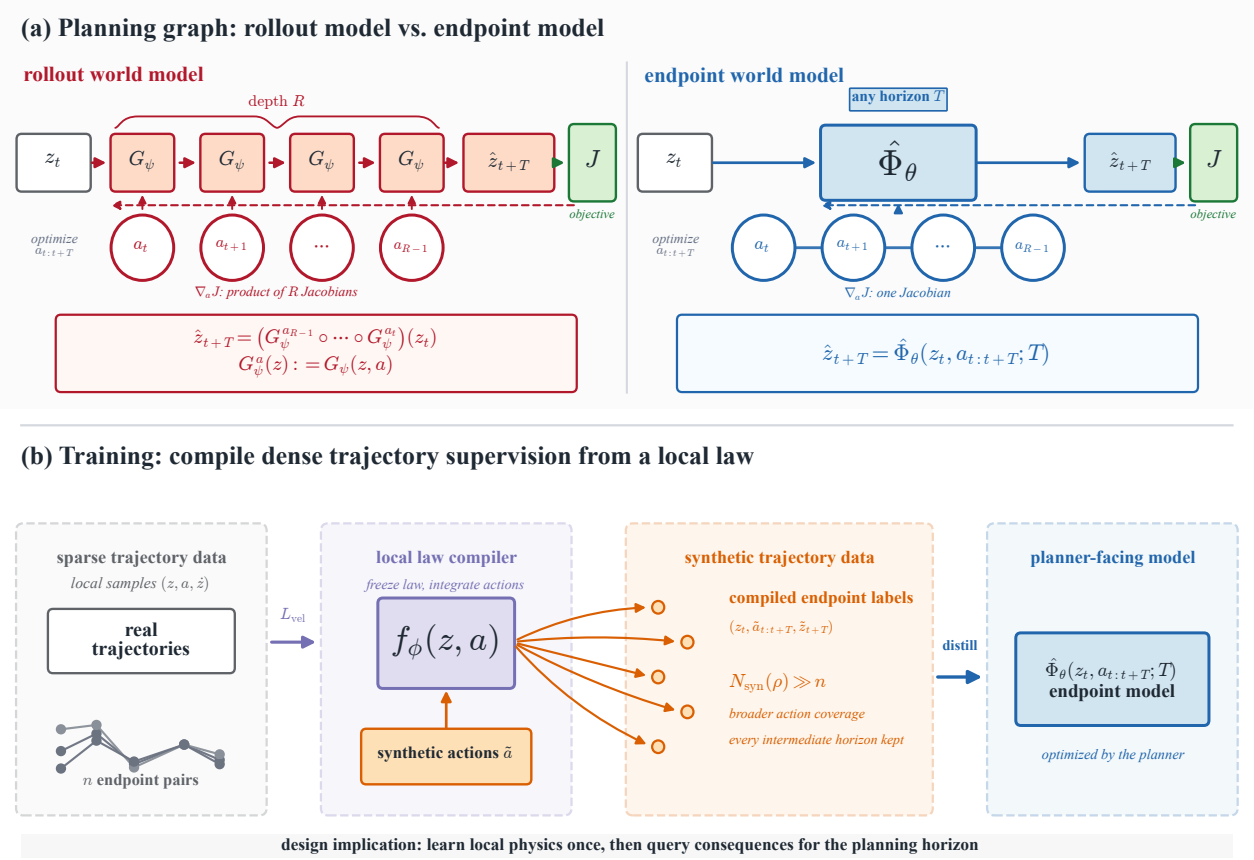

Figure 1. Method overview. **(a)** Planning. The rollout model G_ψ answers a horizon-T query by $R = T/\Delta t$ sequential calls; its action gradient is a product of R Jacobians and is prone to vanishing/exploding (Lemma 3.1). The endpoint world model $\hat{\Phi}_\theta$ maps the initial state and the full action sequence to the endpoint in a single forward pass, so the planner backpropagates through one network Jacobian. The semigroup identity lets the controller query $\hat{\Phi}_\theta$ at a tunable depth, from a single open-loop call to closed-loop replanning. **(b)** Two-stage training. Stage 1 learns a dynamics law f_ϕ from real trajectories by velocity matching. Stage 2 integrates the frozen f_ϕ along synthetic action sequences to compile endpoint targets, keeping every intermediate horizon, which distill the student $\hat{\Phi}_\theta$; the synthetic supervision count $N_{\text{syn}}(\rho) \gg n$ exceeds the paired-data count.

state $z_t := z(t)$ to future state z_{t+T} satisfying

$$\Phi_T^f(z_t, u_{[t,t+T]}) = z_t + \int_t^{t+T} f(z(\tau), u(\tau))\, \mathrm{d}\tau. \quad (1)$$

A result-driven planner searches for a control signal whose predicted endpoint satisfies the desired condition.

Endpoint World Model. We propose learning a horizon-conditioned proxy for the endpoint operator:

$$\hat{\Phi}_\theta(z_t, u_{[t,t+T]}; T) = \hat{z}_{t+T}, \quad (2)$$

where $\hat{z}_{t+T}$ denotes an estimate of z_{t+T}. Practically, the control signal $u_{[t,t+T]}$ is discretized as an action sequence $a_{0:R-1} \in \mathbb{R}^{Rm}$. Under zero-order hold (ZOH), $u(\tau) = a_j$ for $\tau \in [t + j\Delta t,\ t + (j+1)\Delta t)$ with $\Delta t = T/R$. Splines and basis expansions give alternative finite-dimensional parameterizations of the same control signal (Hargraves & Paris, 1987; Kelly, 2017; Ijspeert et al., 2013).

Law-first compilation. We first learn the control law f_ϕ (Chen et al., 2018; Yildiz et al., 2021), then train the endpoint model (2) to match the endpoint operator (1)

$$\Phi_T^{f_\phi}(z_t, u_{[t,t+T]}) := z_t + \int_t^{t+T} f_\phi(z(\tau), u(\tau))\, \mathrm{d}\tau,$$

where the distillation loss (Salimans & Ho, 2022) is

$$\mathcal{L}_{\text{distill}} = \left\| \hat{\Phi}_\theta(z_t, u_{[t,t+T]}; T) - \Phi_T^{f_\phi}(z_t, u_{[t,t+T]}) \right\|^2. \quad (3)$$

Since f_ϕ maps local state-control pairs to instantaneous state changes, it can be integrated along any well-posed queried control signal $\tilde{u}_{[t,t+T]}$, whether observed, synthetic, or recombined. Each such query defines a teacher target $\tilde{z}_{t+T} = \Phi_T^{f_\phi}(z_t, \tilde{u}_{[t,t+T]})$, yielding dense supervision beyond the finite paired trajectory dataset.

Teacher training. We fit f_ϕ by velocity matching against a five-point central finite-difference estimate of the trajectory velocity. At full data this term suffices. Under data scarcity we also roll f_ϕ forward H steps under constant action and supervise every intermediate state against the observed trajectory; we use $H{=}100$ (Section B.6). Velocity matching requires constant-action stencil windows. For varying-action datasets, the integrated trajectory loss alone trains f_ϕ (Section B.6).

Student training. We distill $\hat{\Phi}_\theta$ on synthetic action sequences generated at per-step granularity ($R = T/\Delta t$ actions per sample), with action smoothness controlled by a segment count $B \in [1, R]$ sampled *log-uniformly* per sample. Log-uniform B gives each smoothness octave equal training weight, covering both the smooth regime where planning solutions live and the high-frequency regime that the deployment input distribution exercises. Within each segment the action is sampled uniformly from the control range. The teacher integrates f_ϕ once along the resulting R-step action sequence at $R_{\max}{=}256$ and stores all intermediates: every prefix endpoint $\tilde{z}_{t+r}$, $r \in [1, R_{\max}]$, becomes a distillation target, so one rollout per sample gives continuous horizon coverage.

Optional real-data anchoring. When paired samples $(z_t, u_{[t,t+T]}, T, z_{t+T})$ are available, we regularize $\hat{\Phi}_\theta$ toward the observed true endpoint z_{t+T} rather than the synthetic $\Phi_T^{f_\phi}$ induced by f_ϕ (Song & Dhariwal, 2024):

$$\mathcal{L}_{\text{anchor}} = \left\| \hat{\Phi}_\theta(z_t, u_{[t,t+T]}; T) - z_{t+T} \right\|^2 . \tag{4}$$

The synthetic targets from f_ϕ supply dense coverage over states, horizons, and control signals, while the real-data anchors accuracy on the support of the paired dataset. Direct endpoint regression is the special case where only this term is used and introduced as a baseline below at (7) .

Semigroup-consistency regularization. The endpoint operator induced by any controlled law satisfies the semigroup identity $\Phi_T^f = \Phi_{T-s}^f \circ \Phi_s^f$ for any $0 \le s \le T$. When $\hat{\Phi}_\theta$ is queried at intermediate sub-horizons it should compose consistently with its own one-call prediction. We encode this as an auxiliary loss:

$$\mathcal{L}_{\text{sg}} = \Big\| \hat{\Phi}_\theta(z_t, u_{[t,t+T]}; T) - \hat{\Phi}_\theta\Big(\hat{\Phi}_\theta(z_t, u_{[t,t+s]}; s),\ u_{[t+s,t+T]}; T{-}s \Big) \Big\|^2 , \tag{5}$$

with $s \sim \text{Unif}(0, T)$ and T drawn from the same distribution as the distillation loss. Its weight follows the horizon sampling: Ours samples T continuous-uniformly, so the dense store-all-intermediates coverage already supplies the consistency and the loss is dropped (weight 0.0); the discrete-anchor ablation students (Section B.7) need it to enforce consistency at intermediate horizons absent from the anchor set (weight 0.03). Composition consistency, however obtained, is what lets the controller invoke $\hat{\Phi}_\theta$ at variable depth (Section 3).

Average-velocity parameterization. Alternatively, we parameterize the endpoint model as $\hat{\Phi}_\theta(z_t, u_{[t,t+T]}; T) = z_t + T\,\bar{v}_\theta(z_t, u_{[t,t+T]}; T)$, so the network outputs the average velocity rather than the endpoint itself (Geng et al., 2026). The distillation loss (3) rescales by $1/T$ to $\bar{v}_\theta(z_t, u_{[t,t+T]}; T) \approx \frac{1}{T}\int_t^{t+T} f_\phi(z(\tau), u(\tau))\,\mathrm{d}\tau$. This enforces the identity $\hat{\Phi}_\theta(z_t, u_{[t,t+T]}; 0) = z_t$ and keeps the output well-scaled in the small-horizon limit rather than dominated by z_t.

Reference baselines. We justify our proposed world model $\hat{\Phi}_\theta$ against two reference models, one for planning (Section 3) and one for training (Section 4).

The first is a *rollout model* representing local transition (Hafner et al., 2020; Hansen et al., 2022a) $G_\psi(z_t, a_t) \approx z_{t+\Delta t}$. It answers the planner's finite-horizon query by rolling out for $R = T/\Delta t$ steps:

$$\hat{z}_{t+T} = \big(G_\psi(\,\cdot\,, a_{t+(R-1)\Delta t}) \circ \cdots \circ G_\psi(\,\cdot\,, a_t)\big)(z_t). \tag{6}$$

The second is an *endpoint regressor*, which shares the endpoint supervision but is directly trained on paired trajectory samples $(z_t, u_{[t,t+T]}, T, z_{t+T})$

$$\mathcal{L}_{\text{end}} = \left\| G_\gamma(z_t, u_{[t,t+T]}; T) - z_{t+T} \right\|^2 . \tag{7}$$

3 Planning Advantage

Given a world model, a planner queries predicted future states $\hat{z}_\tau$ to evaluate candidate control signals. Let $r(z, u)$ be an instantaneous reward and $g(z_T)$ a terminal reward. The planner solves the functional optimization

$$\max_{u_{[t,t+T]}} \int_t^{t+T} r\left(\hat{z}(\tau), u(\tau)\right) \mathrm{d}\tau + g\left(\hat{z}_{t+T}\right), \tag{8}$$

to search for the optimal $u_{[t,t+T]}$. Discretize the candidate control signal at the dynamics rate as $a_{0:R-1} \in \mathbb{R}^{Rm}$, where $R = T/\Delta t$. The objective, however, samples only $K \ll R$ reward checkpoints $\tau_j = t + jT/K$, using timestep weight $\Delta\tau = T/K$ and checkpoint action $\bar{a}_j := a_{\lfloor jR/K \rfloor}$. The optimization (8) then becomes finite-dimensional:

$$\max_{a_{0:R-1}} \sum_{j=0}^{K-1} r\left(\hat{z}_{\tau_j}, \bar{a}_j\right) \Delta\tau + g\left(\hat{z}_{t+T}\right). \tag{9}$$

The planning cost has two factors: (i) how expensively the world model answers each checkpoint query $\hat{z}_{\tau_j}$, and (ii) how large the search space over control signals $u_{[t,t+T]}$ is.

Query Cost. For a single query $\hat{z}_{\tau_j}$, the endpoint model evaluates $\widehat{\Phi}_\theta(z_t, u_{[t,\tau_j]}, \tau_j - t)$ once, regardless of the horizon length. The rollout model $G_\psi(z_t, a_t) \approx z_{t+\Delta t}$ returns the same query by $R_j = (\tau_j - t)/\Delta t$ nested calls,

$$\hat{z}_{\tau_j} = \big(G_\psi(\cdot, a_{R_j-1}) \circ \cdots \circ G_\psi(\cdot, a_0)\big)(z_t). \tag{10}$$

For each candidate control signal, the discrete planner in (9) reads K checkpoints $\hat{z}_{\tau_j}$. The rollout model caches intermediate states along the way, so it amortizes to R calls per candidate. The endpoint model directly queries the K required checkpoints. The aggregate per-candidate saving is $R-K$, recovering $R-1$ in the terminal-only case $K=1$.

The dynamics rate Δt must be small enough that G_ψ is Markovian and locally learnable at the underlying frame, motor-command, or decision rate. On the dm_control suite this gives $\Delta t \approx 0.02$s under the standard action-repeat-2 configuration also used by Dreamer and TD-MPC (Tassa et al., 2018; Hafner et al., 2020; Hansen et al., 2022a), so an episode of $T = 20$s already has $R = 10^3$. The number of checkpoints K, however, is set by where reward arrives. Most tasks of interest, e.g., Atari games (Mnih et al., 2015; Bellemare et al., 2013), board games (Schrittwieser et al., 2020), and goal-conditioned robotic manipulation (Andrychowicz et al., 2017), give reward only at score events or task completion, or $K \approx 1$. Hence the saving is essentially the entire rollout length.

Search Structure. For comparison, suppose both models are evaluated on the same grid-level action sequence $a_{0:R-1}$. Let $J_{\widehat{\Phi}}(a_{0:R-1})$ denote the planner's objective (9) under the endpoint model. It depends on this action sequence through one-call queries $\widehat{\Phi}_\theta(z_t, u_{[t,\tau_j]}, \tau_j - t)$. Let $J_G(a_{0:R-1})$ denote the same objective under the rollout model. It depends on the action sequence through the depth-R composition (10). Below we show the search structure of $J_{\widehat{\Phi}}(a_{0:R-1})$.

Lemma 3.1 (Rollout sensitivities contain sequential Jacobian products)**.** *Let the rollout be $\hat{z}_{t+(r+1)\Delta t} = G_\psi(\hat{z}_{t+r\Delta t}, a_r)$ for $r = [0, R_j - 1]$ and $R_j = (\tau_j - t)/\Delta t$. Assume G_ψ is differentiable. For any $s < R_j$, let $A_\ell := \partial_z G_\psi(\hat{z}_{t+\ell\Delta t}, a_\ell)$ and $B_s := \partial_a G_\psi(\hat{z}_{t+s\Delta t}, a_s)$, then*

$$\frac{\partial \hat{z}_{\tau_j}}{\partial a_s} = A_{R_j-1} A_{R_j-2} \cdots A_{s+1} B_s. \tag{11}$$

Proof. The identity follows by repeated application of the chain rule to the rollout recurrence. □

The product gives horizon-dependent sensitivity bounds. Let $d_{s,j} := R_j - s - 1$ be the number of intervening transition Jacobians $\{A_\ell\}_{\ell=s+1}^{R_j-1}$ between action a_s and checkpoint τ_j. If $\|A_\ell\| \le L_G$ and $\|B_s\| \le B_G$ along the rollout,

$$\left\| \frac{\partial \hat{z}_{\tau_j}}{\partial a_s} \right\| \le B_G L_G^{d_{s,j}}.$$

Thus, when $L_G < 1$, sensitivities to earlier actions decay exponentially with the number of intervening rollout steps. Conversely, if $\sigma_{\min}(A_\ell) \ge \mu_G$ and $\sigma_{\min}(B_s) \ge b_G$, then

$$\sigma_{\min}\left(\frac{\partial \hat{z}_{\tau_j}}{\partial a_s}\right) \ge \left(\prod_{\ell=s+1}^{R_j-1} \sigma_{\min}(A_\ell)\right) \sigma_{\min}(B_s) \ge b_G \mu_G^{d_{s,j}}.$$

So, if all intervening A_ℓ satisfy $\sigma_{\min}(A_\ell) \ge \mu_G > 1$, the sensitivity lower bound grows exponentially with depth.

Corollary 3.2 (Rollout planning requires summing all Jacobian-products)**.** *Suppose the rollout objective J_G follows:*

$$J_G(a_{0:R-1}) = \mathcal{J}\left(\hat{z}_{\tau_1}(a_{0:R-1}), \ldots, \hat{z}_{\tau_K}(a_{0:R-1}), a_{0:R-1}\right).$$

Then, for any action a_s,

$$\nabla_{a_s} J_G = \partial_{a_s} \mathcal{J} + \sum_{\substack{j=1 \\ s<R_j}}^{K} \left(\frac{\partial \hat{z}_{\tau_j}}{\partial a_s}\right)^\top \nabla_{\hat{z}_{\tau_j}} \mathcal{J},$$

where each checkpoint sensitivity $\partial \hat{z}_{\tau_j}/\partial a_s$ requires calculating the sequential Jacobian-products in Lemma 3.1.

Proof. Apply the chain rule to the composition of $\mathcal{J}$ with the rollout-generated checkpoints. □

For any action a_s preceding a queried checkpoint τ_j, i.e., $s < R_j$, Lemma 3.1 shows that $\partial \hat{z}_{\tau_j}/\partial a_s$ contains a product of $R_j - s - 1$ transition Jacobians. Corollary 3.2 further shows that gradients of the rollout objective J_G sum over these checkpoint sensitivities. This is the same failure mode as in recurrent networks (Pascanu et al., 2013; Metz et al., 2021); related mitigations are discussed in Section A.

The endpoint objective bypasses this online recurrent graph by querying each checkpoint directly as $\widehat{\Phi}_\theta(z_t, u_{[t,\tau_j]}, \tau_j - t)$. Each checkpoint gradient differentiates through one network Jacobian rather than R_j sequential Jacobians (11). Meanwhile, the K endpoint queries $\{\widehat{\Phi}_\theta(z_t, u_{[t,\tau_j]}, \tau_j - t)\}_{j=1}^K$ share the broadcast input $\mathbf{1}_K \otimes z_t$ and can be resolved in one parallel forward pass. Together, these compress the rollout planner's gradient graph in both depth and width.

Composition depth and replanning. The semigroup identity $\Phi_T^f = \Phi_{T-s}^f \circ \Phi_s^f$ lets a *single* endpoint query be computed at any composition depth $K_{\text{inf}} \in [1, R]$: partition the queried horizon $T = \sum_{k=1}^{K_{\text{inf}}} \tau_k$ and chain K_{inf} depth-1 calls on the model's own predicted intermediate state,

$$\widehat{\Phi}_\theta(z_t, u_{[t,t+T]}; T) \approx \widehat{\Phi}_\theta(\cdot; \tau_{K_{\text{inf}}}) \circ \cdots \circ \widehat{\Phi}_\theta(z_t, u; \tau_1) \tag{12}$$

exact under the identity and tightened by dense continuous-horizon training (or, for the anchored ablation, by $\mathcal{L}_{\text{sg}}$ (5)): $K_{\text{inf}}{=}1$ recovers the single endpoint query, $K_{\text{inf}}{=}R$ the full rollout. The planner backpropagates through K_{inf} Jacobian terms rather than R. Prediction is open-loop, with no environment feedback, and reaches horizons beyond T_{train} without retraining.

Closed-loop MPC replanning is a separate, orthogonal axis: split the *control* horizon into N_{replan} legs, execute each on the environment, and replan from the *realized* state. The two nest: a plan with N_{replan} legs at composition depth K_{inf} issues $N_{\text{replan}}\,K_{\text{inf}}$ model evaluations per gradient step, and each leg needs accuracy only at its sub-horizon $\le T_{\text{train}}$. Sub-goal and waypoint planners (Section A) share this depth-1-per-stage structure; rollout-within-stage hierarchies (TD-MPC's H-step lookahead (Hansen et al., 2022b), Dreamer's imagined rollouts (Hafner et al., 2020)) retain the depth-k Jacobian-product pathology inside each stage.

4 Training Advantage

Both direct endpoint regression and law-first compilation share the same supervision form. They differ in the target distribution. For $\Delta t_i = T_i/R_i$, let the trajectory dataset be

$$\mathcal{D}_{\text{traj}} = \{(z_0^{(i)}, a_0^{(i)}, z_1^{(i)}, \dots, a_{R_i-1}^{(i)}, z_{R_i}^{(i)}, T_i)\}_{i=1}^n.$$

Direct endpoint regression fits $G_\gamma(z_0^{(i)}, a_{0:R_i-1}^{(i)}; T_i)$ to the observed endpoint $z_{R_i}^{(i)}$ via the tuples $(z_0^{(i)}, a_{0:R_i-1}^{(i)}, T_i, z_{R_i}^{(i)})$. Such supervision must cover the high-dimensional query space of states, horizons, and action sequences.

Law-first compilation generates supervision by integrating a learned control law f_ϕ (1). The law is trained on the $n_{\text{loc}} = \sum_{i=1}^n R_i$ samples $\{(z_r^{(i)}, a_r^{(i)}) \mapsto (z_{r+1}^{(i)} - z_r^{(i)})/\Delta t_i\}_{i,r}$. Given f_ϕ, any well-posed action sequence $\tilde a_{0:R_i-1}$ defines a control signal $\tilde u_{[0,T_i]}$ and hence a teacher target $\tilde z_{R_i} = \Phi_{T_i}^{f_\phi}(z_0^{(i)}, \tilde u_{[0,T_i]})$. This target trains the student $\hat\Phi_\theta$ via distillation (3). The queried action sequence $\tilde a_{0:R_i-1}$ may be observed, synthetic, or recombined, and need not be paired with an observed endpoint. In the following paragraphs, we show law-first compilation dominates direct endpoint regression on $\mathcal{D}_{\text{traj}}$ in two ways: (i) expanding the effective supervision and (ii) shrinking the hypothesis class.

Scale of Synthetic Supervision We now quantify the synthetic supervision for $\hat\Phi_\theta$. Let the local training support be $\mathcal{D}_{\text{loc}} = \{(z_r^{(i)}, a_r^{(i)}) : i = 1, \dots, n\}_{r=0}^{R_i-1}$.

Proposition 4.1 (Synthetic supervision count)**.** *Fix $\rho > 0$. For state z, let $\mathcal{A}_\rho(z) := \{a : \text{dist}((z,a), \mathcal{D}_{\text{loc}}) \le \rho\}$. For anchor $z_0^{(i)}$, let $\tilde z_0 := z_0^{(i)}$ and $\tilde z_{r+1} := \tilde z_r + \Delta t_i\, f_\phi(\tilde z_r, \tilde a_r)$. The usable number of synthetic endpoint queries is $N_{\text{syn}}(\rho) := \sum_{i=1}^n |\mathcal{V}_\rho^{(i)}|$, where $\mathcal{V}_\rho^{(i)}$ is a valid action sequence set defined by*

$$\mathcal{V}_\rho^{(i)} := \{\tilde a_{0:R_i-1} : \tilde a_r \in \mathcal{A}_\rho(\tilde z_r)\ \forall r = 0, \dots, R_i - 1\}.$$

An action sequence is valid as each rollout pair $(\tilde z_r, \tilde a_r)$ stays within ρ of local training support, so f_ϕ generalizes along the synthetic trajectory and the endpoint target $\tilde z_{R_i}$ is reliable. The valid sets $\{\mathcal{V}_\rho^{(i)}\}_{i=1}^n$ induce a mixture over action sequences around the observed anchors. For example, sample an anchor $i \sim \text{Unif}(\{1, \dots, n\})$, then sample $\tilde a_{0:R_i-1}$ from a distribution supported on $\mathcal{V}_\rho^{(i)}$. Thus $N_{\text{syn}}(\rho)$ measures the size of the upsampled support for synthetic supervision. A ρ-net argument (Vershynin, 2018) on $\mathcal{A}_\rho$ gives $|\mathcal{V}_\rho^{(i)}| \gtrsim (\rho/\epsilon)^{d_a R_i}$ at resolution ϵ, hence $N_{\text{syn}}(\rho)$ grows exponentially in horizon and $N_{\text{syn}}(\rho) \gg n$ whenever $\rho > \epsilon$.

Smaller Hypothesis Class The student endpoint model $\hat\Phi_\theta$ distilled from the local teacher f_ϕ shares the same architecture and query space with the endpoint regressor G_γ. The statistical difference originates from the target. Denote $G_\gamma^{a,b}(\cdot) := G_\gamma(\cdot, u_{[a,b]}; b-a)$. Direct endpoint regression learns an unconstrained map $G_\gamma^{t,t+T}(z_t) \approx z_{t+T}$, but the loss does not enforce the semigroup identity

$$G_\gamma^{t,t+T} \not\equiv G_\gamma^{s,t+T} \circ G_\gamma^{t,s}, \qquad t < s < t+T.$$

In contrast, law-first compilation generates targets by integrating the learned local f_ϕ. So all teacher labels satisfy

$$\Phi_T^{f_\phi} = \Phi_{t+T-s}^{f_\phi} \circ \Phi_{s-t}^{f_\phi}, \qquad t < s < t+T.$$

Thus compilation supplies supervision constrained to a semigroup-consistent target family.

Let $\mathcal{H}_{\text{end}}$ be the hypothesis class of general horizon-conditioned endpoint maps and $\mathcal{H}_{\text{flow}} = \{\Phi^f : f \in \mathcal{F}_{\text{dyn}}\}$ the subclass induced by f. Since integrating f imposes semigroup consistency, $\mathcal{H}_{\text{flow}} \subsetneq \mathcal{H}_{\text{end}}$. Thus law-first compilation restricts the teacher class to $\mathcal{H}_{\text{flow}}$ and trains the student $\hat\Phi_\theta$ on semigroup-constrained supervision.

Combining the expanded supervision with ($N_{\text{syn}}(\rho) \gg n$) and shrunken hypothesis class ($\mathcal{H}_{\text{flow}} \subsetneq \mathcal{H}_{\text{end}}$), standard excess-risk bounds (e.g., Rademacher / VC) for law-first compilation are $\mathcal{O}(\text{cap}(\mathcal{H}_{\text{flow}})/N_{\text{syn}}(\rho))$, strictly smaller than $\mathcal{O}(\text{cap}(\mathcal{H}_{\text{end}})/n)$ for direct endpoint regression.

5 Empirical Evidence

We evaluate on Pendulum-v1 (Brockman et al., 2016) with timestep $\Delta t = 0.02$ s. The training set contains 128 trajectories of 300 steps (6 s each); the test set contains 64 trajectories. Section 5.1 reports prediction MSE (Table 1) and terminal-cost planning (Table 2) across $T \in \{1, 2, 3, 4, 5\}$ s on the same model checkpoints and baseline set. The student is a single any-horizon operator: each teacher rollout keeps its full trajectory, every horizon is a distillation target, and the supervision horizon is drawn continuous-uniformly over $T \in [0.08, 5.12]$ s; dense coverage replaces the semigroup loss (weight 0, Section B.7). Tables report the canonical training seed; error bars, where shown, are mean±std over three training seeds. Section 5.2 and the appendix ablations report prediction MSE over $T = 5.12$ s (256 steps). We compare our endpoint model $\hat\Phi_\theta$ trained in (3), against an *endpoint regressor* G_γ (7) and a *rollout model* G_ψ fit at Δt and rolled out $R = T/\Delta t$ times (6). Architecture, hyperparameters, planning protocol, and ablations are deferred to Section B.

5.1 Prediction and planning across horizons

The planning task is to drive the pendulum to the upright equilibrium $(\theta, \dot{\theta}) = (0, 0)$ from 30 random initial states. The plan is a per-step action sequence at every env step ($R = T/\Delta t$ actions) and we report median terminal cost C and the fraction of plans reaching $C < 2$ (success). Endpoint models optimize the action sequence by Adam gradient descent on the planning cost. The rollout model has no usable action gradients (Lemma 3.1), so we plan with cross-entropy method and report an additional gradient-descent ablation row (Section B.3). The MSE protocol is the per-step varying-action input from Table 3, evaluated at each T.

Table 1. Endpoint MSE on per-step varying-action test sequences, n=256, deployment-distribution input. Same protocol as Table 3 extended across horizons. Cells: mean / median MSE at the given T. K_{inf} is the open-loop composition depth (the model chained on its own predicted intermediate state, no environment feedback); this is pure prediction, orthogonal to the closed-loop replanning count N_{replan} (Section B.4). Bold marks the best mean and best median in each column.

Model	T=1	T=2	T=3	T=4	T=5
G_γ^{scalar}	0.85/0.072	2.71/0.188	4.27/0.577	7.97/1.045	8.97/2.080
G_γ^{seq}	25.43/19.47	8.74/1.101	21.45/17.93	19.61/8.840	26.80/19.15
G_ψ (rollout)	0.51/0.009	3.25/0.044	4.39/0.144	5.43/0.439	6.38/1.056
Ours $\hat{\Phi}_\theta$ (K_{inf}=1)	**0.13**/0.009	**1.73**/0.034	**2.74**/0.107	3.45/0.199	**4.94**/0.699
Ours $\hat{\Phi}_\theta$ (K_{inf}=2)	0.20/0.003	2.14/0.020	3.51/0.045	**3.39**/0.105	4.98/0.203
Ours $\hat{\Phi}_\theta$ (K_{inf}=4)	0.41/0.003	2.03/**0.009**	3.49/0.046	4.74/0.084	6.00/0.146
Ours $\hat{\Phi}_\theta$ (K_{inf}=8)	0.36/**0.002**	1.80/0.010	3.09/**0.024**	4.04/**0.080**	4.96/**0.118**

Table 2. Any-horizon planning at $T \in \{1, 2, 3, 4, 5\}$ s. Cells: median terminal cost / reach rate ($C < 2$) over 30 episodes. Gradient planners use $G = 50$ Adam steps; sampling planners (CEM and MPPI) use $N_{\text{cand}} = 128$ candidates over $N_{\text{iter}} = 4$ iterations. Compute is forward calls per plan: for G_ψ it scales with $R = T/\Delta t$, constant otherwise (closed form: Table 5). One trained $\hat{\Phi}_\theta$ underlies the four Ours rows, each a single open-loop plan over the full horizon scored at composition depth K_{inf} (Table 1), with per-row compute 50 K_{inf}. The orthogonal closed-loop replanning axis N_{replan} and the offline-trained RL controllers (PETS, IQL, CQL) are deferred to Section B.4. Bold cells mark the best non-oracle median or reach in each column.

Model	Planner	Compute	T=1	T=2	T=3	T=4	T=5
Oracle	CEM	4,096	1.86/60%	0.71/73%	3.78/30%	0.76/80%	2.15/47%
Oracle	gradient	400	0.82/80%	0.42/97%	0.17/100%	0.09/100%	0.22/100%
G_γ^{scalar}	gradient	400	1.33/70%	0.92/83%	1.58/53%	0.63/87%	0.69/77%
G_γ	CEM	$\approx 25\text{k} \cdot T$	1.70/57%	1.33/60%	6.56/27%	3.39/33%	3.87/33%
G_ψ	MPPI	$\approx 25\text{k} \cdot T$	2.29/43%	1.89/53%	3.30/30%	4.22/33%	3.59/37%
G_ψ	gradient	$\approx 2.5\text{k} \cdot T$	0.88/80%	0.52/77%	0.46/70%	0.49/73%	0.87/67%
Ours $\hat{\Phi}_\theta$ (K_{inf}=1)	gradient	50	0.79/**83**%	0.40/97%	1.17/73%	2.66/40%	3.29/33%
Ours $\hat{\Phi}_\theta$ (K_{inf}=2)	gradient	100	**0.78**/**83**%	0.34/**100**%	0.46/**93**%	0.30/80%	0.63/83%
Ours $\hat{\Phi}_\theta$ (K_{inf}=4)	gradient	200	0.92/80%	**0.32**/93%	0.38/90%	0.33/**97**%	0.35/83%
Ours $\hat{\Phi}_\theta$ (K_{inf}=8)	gradient	400	0.93/80%	0.33/93%	**0.17**/**93**%	**0.26**/93%	**0.33**/**90**%

Result (MSE, Table 1). A single distilled operator wins every column on both mean and median, and each baseline fails the way Section 4 predicts. The G_γ regressors fit endpoints in the unconstrained class $\mathcal{H}_{\text{end}}$ and cannot leave their training distribution: G_γ^{seq} collapses on per-step inputs. G_ψ, the strongest baseline, compounds one-step error over R=$T/\Delta t$ sequential calls; Ours matches or beats it with one. The K_{inf} rows trace the depth axis of (12): the single call avoids compounding and takes the best means, composition tightens the medians, six-fold at T=5 s. The margin is seed-robust: mean 5.63 ± 1.00 at T=5 s against the anchored ablation's 7.18 ± 1.69 (Section B.7).

Result (any-horizon planning, Table 2). One network plans every horizon, and the winning composition depth tracks Table 1. At that depth Ours beats the strongest non-oracle baseline, G_ψ gradient, on median and reach at every horizon with 31× less compute at T=5 s (124× at depth 2), and matches the true-dynamics Oracle's gradient planner on median through T=3 s. This is the gradient-graph compression of Section 3: a G_ψ gradient step backpropagates through an R=$T/\Delta t$-deep rollout, Ours through at most eight composed calls; the sampling planners (CEM, MPPI) trail the gradient planners for both G_ψ and the Oracle. Dense continuous-horizon training keeps composed predictions consistent, so depth is a pure inference-time choice. Closed-loop, the same network plans zero-shot where the offline RL controllers need 10^5 task-specific gradient steps (Section B.4).

Anchored ablation. The operator is valid at any $T \in [0.08, 5.12]$ s, not only the integer columns above: median endpoint MSE stays ≤ 0.11 at every $T \in \{0.5, 1, 1.5, 2, 2.5, 3\}$ s, enabling step-level MPC. Replacing the continuous-uniform horizon sampling with nine discrete anchors (with the semigroup loss at 0.03, which sparse sampling requires) yields the anchored ablation: worse on every headline axis, wider seed-variance, and off-anchor queries fail outright (Section B.7).

5.2 Data efficiency under scarcity

We probe data efficiency (cf. Proposition 4.1) by comparing Ours trained on all 128 trajectories against a Scarce variant retrained on 10% of them, with the anchored ablation pair trained on the same splits; Table 3 reports held-out endpoint MSE against the per-step baselines.

Table 3. Endpoint MSE at $T = 5.12$ s on $n = 256$ test sequences with action varying at every env step. Cells: mean / median; our students mean±std over three training seeds, baselines trained once. The anchored pair is the discrete-anchor ablation (Section B.7). Teacher recipe and scarcity diagnostic: Section B.6.

	Mean	Median
G_γ^{scalar} (constant)	9.22	1.651
G_γ^{seq} (per-step)	25.41	17.095
G_ψ (per-step rollout)	6.77	0.991
Ours (any-horizon)	6.57 ± 1.64	2.71 ± 2.22
Ours Scarce (10%)	8.74 ± 1.51	4.67 ± 2.98
Anchored (ablation)	8.55 ± 2.19	6.09 ± 4.52
Anchored Scarce (10%)	10.38 ± 3.76	7.69 ± 6.36

Result (MSE, Table 3). This is Proposition 4.1 in operation: the distillation count $N_{\text{syn}}(\rho)$ is set by the synthetic compilation, not the real-trajectory count, so a 10× data cut costs Ours 1.3× on mean and the Scarce student still beats both full-data G_γ baselines. Ours and G_ψ both re-

Table 4. Planning quality of the Scarce (10% data) students, open-loop single-call (K_{inf}=1), per-step actions. Cells: median terminal cost / reach % over 30 episodes; 50 forward calls per plan; Ours row mean±std over three training seeds, anchored row canonical seed. Same task and protocol as Table 2.

Method	T=1	T=2	T=3	T=5
Ours Scarce (10%)	1.25±0.21/70±6%	1.31±0.62/70±12%	3.72±0.64/31±7%	4.75±1.33/24±5%
Anchored Scarce (ablation)	1.23/70%	1.24/63%	1.17/70%	7.29/20%

alize $N_{\text{syn}}(\rho) \gg n$ inside the semigroup-consistent class $\mathcal{H}_{\text{flow}} \subsetneq \mathcal{H}_{\text{end}}$ and land at matched mean, G_ψ with the lowest median; what the flow map adds at matched MSE is any-horizon queries, composition, and one-call planning gradients. The G_γ baselines, confined to the n real pairs in $\mathcal{H}_{\text{end}}$, fail in both forms: the scalar interface cannot represent per-step action variation, the sequence form cannot leave its constant-action training distribution. The anchored ablation is worse and noisier on every row.

Result (planning, Table 4). Scarcity costs the long horizons first: the Scarce student plans $T \leq 2$ s at near 70% reach, degrades from T=3 s where the anchored Scarce holds, and both fail at T=5 s. Under scarce data the anchor-plus-semigroup prior buys long-horizon planning that dense coverage must instead learn from data, an advantage that disappears at full data.

6 Concluding Remarks

We present synthetic data compilation for rollout-free trajectory planning. By replacing recurrent rollout with endpoint consequence queries, the method improves planning cost and action-gradient structure (Section 3). By compiling endpoint targets from a learned dynamics teacher, it improves supervision coverage under paired-data scarcity (Section 4). Experiments support both advantages (Section 5).

References

Andrychowicz, M., Wolski, F., Ray, A., Schneider, J., Fong, R., Welinder, P., McGrew, B., Tobin, J., Pieter Abbeel, O., and Zaremba, W. Hindsight experience replay. *Advances in neural information processing systems*, 30, 2017.

Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. The arcade learning environment: An evaluation platform for general agents. *Journal of artificial intelligence research*, 47:253–279, 2013.

Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., and Zaremba, W. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.

Chane-Sane, E., Schmid, C., and Laptev, I. Goal-conditioned reinforcement learning with imagined subgoals. In *International conference on machine learning*, pp. 1430–1440. PMLR, 2021.

Chen, R. T. Q., Rubanova, Y., Bettencourt, J., and Duvenaud, D. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems*, 2018.

Chua, K., Calandra, R., McAllister, R., and Levine, S. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. *Advances in neural information processing systems*, 31, 2018.

Finn, C. and Levine, S. Deep visual foresight for planning robot motion. In *2017 IEEE international conference on robotics and automation (ICRA)*, pp. 2786–2793. IEEE, 2017.

Geng, Z., Deng, M., Bai, X., Kolter, Z., and He, K. Mean flows for one-step generative modeling. *Advances in Neural Information Processing Systems*, 38:75460–75482, 2026.

Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., and Davidson, J. Learning latent dynamics for planning from pixels. In *International conference on machine learning*, pp. 2555–2565. PMLR, 2019.

Hafner, D., Lillicrap, T., Ba, J., and Norouzi, M. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020.

Hansen, N., Wang, X., and Su, H. Temporal difference learning for model predictive control. In *International Conference on Machine Learning*, 2022a.

Hansen, N., Wang, X., and Su, H. Temporal difference learning for model predictive control. *arXiv preprint arXiv:2203.04955*, 2022b.

Hargraves, C. R. and Paris, S. W. Direct trajectory optimization using nonlinear programming and collocation. *Journal of Guidance, Control, and Dynamics*, 10(4):338–342, 1987. doi: 10.2514/3.20223.

Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

Hou, X., Huang, X., and Perdikaris, P. Cfo: Learning continuous-time pde dynamics via flow-matched neural operators. *arXiv preprint arXiv:2512.05297*, 2025.

Ijspeert, A. J., Nakanishi, J., Hoffmann, H., Pastor, P., and Schaal, S. Dynamical movement primitives: Learning attractor models for motor behaviors. *Neural Computation*, 25(2):328–373, 2013. doi: 10.1162/NECO_a_00393.

Janner, M., Li, Q., and Levine, S. Offline reinforcement learning as one big sequence modeling problem. *Advances in neural information processing systems*, 34: 1273–1286, 2021.

Janner, M., Du, Y., Tenenbaum, J. B., and Levine, S. Planning with diffusion for flexible behavior synthesis. *arXiv preprint arXiv:2205.09991*, 2022.

Jurgenson, T., Avner, O., Groshev, E., and Tamar, A. Subgoal trees a framework for goal-based reinforcement learning. In *International conference on machine learning*, pp. 5020–5030. PMLR, 2020.

Kelly, M. An introduction to trajectory optimization: How to do your own direct collocation. *SIAM Review*, 59(4): 849–904, 2017. doi: 10.1137/16M1062569.

Korda, M. and Mezić, I. Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control. *Automatica*, 93:149–160, 2018.

Kostrikov, I., Nair, A., and Levine, S. Offline reinforcement learning with implicit q-learning. In *International Conference on Learning Representations*, 2022.

Kumar, A., Zhou, A., Tucker, G., and Levine, S. Conservative q-learning for offline reinforcement learning. *Advances in Neural Information Processing Systems*, 33: 1179–1191, 2020.

Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., and Anandkumar, A. Fourier neural operator for parametric partial differential equations. *arXiv preprint arXiv:2010.08895*, 2020.

Lin, H., Xu, Y.-Y., Sun, Y., Zhang, Z., Li, Y.-C., Jia, C., Ye, J., Zhang, J., and Yu, Y. Any-step dynamics model improves future predictions for online and offline reinforcement learning. *arXiv preprint arXiv:2405.17031*, 2024.

Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., and Le, M. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.

Liu, Q. Rectified flow: A marginal preserving approach to optimal transport. *arXiv preprint arXiv:2209.14577*, 2022.

Lu, L., Jin, P., Pang, G., Zhang, Z., and Karniadakis, G. E. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. *Nature machine intelligence*, 3(3):218–229, 2021.

Metz, L., Freeman, C. D., Schoenholz, S. S., and Kachman, T. Gradients are not all you need. *arXiv preprint arXiv:2111.05803*, 2021.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. Human-level control through deep reinforcement learning. *nature*, 518(7540): 529–533, 2015.

Pascanu, R., Mikolov, T., and Bengio, Y. On the difficulty of training recurrent neural networks. In *International conference on machine learning*, pp. 1310–1318. Pmlr, 2013.

Salimans, T. and Ho, J. Progressive distillation for fast sampling of diffusion models. *arXiv preprint arXiv:2202.00512*, 2022.

Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., Guez, A., Lockhart, E., Hassabis, D., Graepel, T., et al. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839): 604–609, 2020.

Song, Y. and Dhariwal, P. Improved techniques for training consistency models. In *International Conference on Learning Representations*, volume 2024, pp. 15078–15097, 2024.

Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., and Poole, B. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020.

Tassa, Y., Doron, Y., Muldal, A., Erez, T., Li, Y., Casas, D. d. L., Budden, D., Abdolmaleki, A., Merel, J., Lefrancq, A., et al. Deepmind control suite. *arXiv preprint arXiv:1801.00690*, 2018.

Vershynin, R. *High-dimensional probability: An introduction with applications in data science*, volume 47. Cambridge university press, 2018.

Yildiz, C., Heinonen, M., and Lahdesmaki, H. Continuous-time model-based reinforcement learning. In *International Conference on Machine Learning*, 2021.

A Related Work and Discussion

Rollout-based MBRL and recurrent planning. Long-horizon rollout depth is a standard difficulty in model-based RL. Backpropagation through a recurrent computation graph produces products of transition Jacobians (Lemma 3.1), which can cause vanishing or exploding sensitivities (Pascanu et al., 2013). Existing world-model methods mitigate the recurrent rollout structure rather than remove it. PlaNet plans by CEM in a learned recurrent latent space, regularized by latent overshooting for multi-step consistency (Hafner et al., 2019). Dreamer learns actors and critics by backpropagating through imagined latent rollouts of a compact recurrent dynamics model (Hafner et al., 2020). TD-MPC truncates planning horizons with a learned terminal value, reducing rollout depth at planning time (Hansen et al., 2022b). All three keep recurrent composition in the model loop. Our formulation removes the recurrent rollout itself. For terminal or sparse-reward objectives, the planner queries $\widehat{\Phi}_\theta(z_t, u_{[t,t+\tau]}; \tau)$ directly instead of traversing the recurrent graph (Section 3)

Hierarchical and decomposed planning. Horizon decomposition splits into rollout-within-stage and depth-1-per-stage. Rollout-within-stage schemes shorten the per-stage horizon but each stage is itself a multi-step rollout: TD-MPC's H-step lookahead (Hansen et al., 2022b), Dreamer's imagined rollouts (Hafner et al., 2020). Each stage incurs the depth-k Jacobian-product pathology (Lemma 3.1). Depth-1-per-stage schemes replace each stage with a single-shot prediction, as in sub-goal and waypoint reasoners (Jurgenson et al., 2020; Chane-Sane et al., 2021). Our compositional inference (12) is depth-1-per-stage with three differences from those reasoners: each call ingests a full action sub-sequence rather than a goal state (continuous controls, end-to-end gradient); composability is trained in, from dense store-all-intermediates coverage for the continuous-horizon student and from the explicit semigroup loss (5) for the discrete-anchor ablation; and the composition depth K_{inf} and replanning count N_{replan} are independent controller-time parameters. Diffusion and consistency planners (Janner et al., 2022; Song et al., 2020) are also depth-1-per-stage but compose in generative time, not physical time.

Dynamics and operator learning. A broad line of work learns dynamical laws or solution operators rather than fixed-step transition maps. Neural ODEs and continuous-time MBRL parameterize instantaneous vector fields instead of immediate next-state predictors (Chen et al., 2018; Yildiz et al., 2021). Koopman-based methods learn lifted linear predictors for nonlinear controlled dynamics and have been used inside MPC (Korda & Mezić, 2018). Neural operators, including DeepONet and Fourier neural operators, learn mappings between function spaces and amortize PDE solution operators across initial conditions, parameters, or forcing functions (Lu et al., 2021; Li et al., 2020). These works primarily target prediction, simulation, system identification, or control within a learned or partially specified dynamical system. We use dynamics as a learned representation of controlled state change without a known simulator or symbolic model. The learned local law can then be recombined across supported action sequences to compile finite-horizon endpoint supervision (Sections 2 and 4).

State-change modeling across generative and physical time. Diffusion and score-based models learn denoising scores or reverse-time dynamics and generate samples through iterative solvers over diffusion time (Song et al., 2020; Ho et al., 2020). Flow matching and rectified flow instead learn velocity fields along probability paths (Lipman et al., 2023; Liu, 2022). One-step flow models replace instantaneous velocity with average state change to reduce inference-time solver depth (Geng et al., 2026). Although these methods are usually formulated in generative time, related physical-time work applies flow-matching ideas to continuous-time PDE dynamics by fitting evolution fields rather than autoregressive transitions (Hou et al., 2025). Our work uses the same state-change viewpoint in controlled dynamics, but the output is a planner-facing endpoint predictor rather than a generative sampler (Section 2).

Trajectory prediction and supervision scarcity. Action-conditioned visual foresight, any-step dynamics, and trajectory-level generative models move beyond immediate next-state prediction by modeling longer-horizon futures or full trajectories (Finn & Levine, 2017; Lin et al., 2024; Janner et al., 2021; 2022). Statistically, these approaches learn maps or distributions over high-dimensional trajectory objects whose dimension grows with horizon and action-sequence length. Yet valid trajectories are highly structured: they are consequences of a local controlled law, not arbitrary state-action sequences. Directly fitting future or trajectory maps from paired trajectory data can therefore spend capacity on a large joint object rather than the lower-dimensional dynamical rule that generates it. Law-first compilation uses this structure explicitly by compiling supported synthetic endpoint targets from the learned local law, improving trajectory coverage while constraining targets to dynamics-induced consequences (Section 4 and Proposition 4.1).

B Experimental Setup and Ablations

This section contains architecture, training-recipe, planning-protocol, and ablation details deferred from Section 5. All experiments use the same Pendulum-v1 dataset of 128 training trajectories and 64 held-out test trajectories, each 300 steps long. Trajectories are generated with a constant uniform control in $[-2, 2]$ and integration step $\Delta t = 0.02$ s. Test trajectories use a different random seed.

B.1 Models

Control law f_ϕ. A multilayer perceptron (MLP) with 4 layers of width 256 and SiLU activations, mapping $(z, u) \mapsto \dot{z}$ with no time input. Trained by velocity matching against 5-point central finite differences (`velocity_matching_central_diff_order: 5`) on the trajectory dataset; the central-difference target reduces the velocity-matching bias from $\mathcal{O}(\Delta t)$ to $\mathcal{O}(\Delta t^4)$. Integrated with RK4 over 8 substeps for distillation queries.

Endpoint regressor G_γ^{scalar}. A multilayer perceptron (MLP) with 4 layers of width 512 and SiLU activations, mapping $(z_t, u, h) \mapsto v$ in the average-velocity parameterization $\hat{z}_{t+h} = z_t + h\, v$. Trained on observed (z_t, u, h, z_{t+h}) tuples sampled from the training trajectories with h drawn uniformly from $[1, 250]$ environment steps. Assumes the control u is held constant over $[t, t + h]$.

Endpoint regressor G_γ^{seq}. Same attention architecture as $\hat{\Phi}_\theta$ below, but trained on real observed endpoint tuples $(z_t, u_{[t,t+T]}, T, z_{t+T})$ instead of synthetic teacher rollouts. Because the raw trajectories are constant-control, every observed action sequence is degenerate (K copies of one scalar action). Isolates the contribution of synthetic supervision from the contribution of the sequence interface.

Rollout model G_ψ. A multilayer perceptron (MLP) with 3 layers of width 128, mapping $(z_t, u) \mapsto \hat{z}_{t+\Delta t}$ at a fixed step $\Delta t = 0.02$ s. At query time the model is unrolled $R = T/\Delta t$ base steps under zero-order hold of the candidate control (Section B.3).

Our endpoint world model $\hat{\Phi}_\theta$. A Transformer encoder over the action-chunk tokens $\{(u_r, h_r)\}_{r=0}^{R-1}$ produces a context vector that is concatenated with the initial state z_t and passed to an average-velocity head with 4 layers of width 512. The encoder uses 2 self-attention blocks with hidden dimension 128, 4 heads, and feed-forward width 512. One forward pass produces the predicted endpoint $\hat{z}_{t+T}$ regardless of R.

B.2 Two-stage training

Stage 1 (law). f_ϕ is trained by 5-point central-difference velocity matching on the trajectory dataset. We use batch size 256, 6000 optimization steps, Adam with learning rate 10^{-3} and weight decay 10^{-6}, gradient clipping at 1.0.

Stage 2 ($\hat{\Phi}_\theta$). The student is trained against teacher endpoints $\Phi_T^{f_\phi}(z_t, \tilde{u}_{[t,t+T]})$ obtained by integrating the frozen f_ϕ under synthetic action sequences. We do not use observed endpoints during stage 2. Batch size 256, 6000 optimization steps, Adam with learning rate $7 \cdot 10^{-4}$ and weight decay 10^{-6}, gradient clipping at 1.0.

For each training step we draw one synthetic action sequence over the full $R_{\max} = 256$ env steps ($T = 5.12$ s) and integrate the frozen f_ϕ once with the store-all-intermediates compilation: every prefix length $r \in [1, 256]$ is kept as a $(z_t, u_{[t,t+r]}, r, z_{t+r})$ distillation target, giving the compilation continuous horizon coverage (the student draws its supervision horizon continuous-uniformly from this pool; the anchored ablation of Section B.7 draws it at nine discrete anchors). The sequence is piecewise constant in B segments of equal length, with B drawn log-uniformly over $[1, R]$: $B = 1$ recovers the constant-control distribution of the real trajectories, $B = R$ varies the control at every env step, and the log-uniform draw weights each smoothness octave equally. Each segment's control value is sampled independently from the training-data action pool. The teacher integrates f_ϕ sequentially with 8 RK4 substeps per env step, and the integrated rollout stays near states where f_ϕ is accurate.

B.3 Planning protocol

Gradient planning (endpoint models). For an endpoint operator $\hat{\Phi}_\theta$ and the endpoint regressor G_γ, the action sequence $a = (a_0, \ldots, a_{R-1}) \in \mathbb{R}^{Rm}$ is a parameter tensor. Adam optimizes the terminal cost C for 50 steps at learning rate 0.1 with initial standard deviation 0.3. Each step performs one forward and one backward pass through the model. Our endpoint world model $\hat{\Phi}_\theta$ takes the full sequence in one call, so each step costs one forward call (50 total). The scalar endpoint regressor G_γ^{scalar} cannot ingest the per-step sequence; it scores the plan over K=8 constant-action control chunks and chains K calls per step ($K \cdot 50 = 400$ total).

CEM (rollout baseline). The rollout model G_ψ does not admit a usable action gradient in the recurrent form (Lemma 3.1). We plan with cross-entropy method: a Gaussian over the $R = T/\Delta t$ per-step control sequence, 128 candidates per iteration, 4 iterations, elite fraction 0.1, initial standard deviation 1.0. Each candidate is scored by unrolling G_ψ for all R micro-steps. Total forward calls per plan is candidates $\times$ iterations $\times$ (R micro-steps per candidate).

MPPI (rollout baseline). We also report a Model-Predictive Path Integral baseline for G_ψ. MPPI uses the same per-step Gaussian and the same 128×4 sampling budget as CEM, but updates the Gaussian by the exponentially-weighted mean and variance of the candidate costs with temperature $\lambda = 1.0$ rather than by the top-N elite mean. This standard reference planner rules out a weak-CEM artifact behind the rollout-model wedge in Table 2.

Gradient ablation on G_ψ. For the gradient-descent rows of G_ψ in Table 2, we apply the same Adam recipe through the chained R-step rollout. This pays $R \cdot G$ forward calls per plan and tests whether the gradient pathology of Lemma 3.1 is empirically severe.

Compute counts in Table 2 as a closed form. Forward calls per plan are fully determined by the planner's parameters and each model's per-evaluation cost. Let $G = 50$ be the gradient planner's Adam steps, $N_{\text{cand}} = 128$ and $N_{\text{iter}} = 4$ the CEM candidates per iteration and iteration count, $K = 8$ the control-chunk count that the scalar-interface baselines (G_γ^{scalar} and the Oracle) are restricted to (constant action per chunk; this is a planner discretization, not the retired 8-chunk evaluation input), $K_{\text{inf}} \in \{1, 2, 4, 8\}$ the open-loop composition depth of Ours, and $R = T/\Delta t$ the env micro-steps over the horizon ($\Delta t = 0.02$ s, so $R = 100$ at $T = 2$ s). Ours and G_ψ instead plan the full per-step R-action sequence. One "forward call" is one invocation of `model.predict`, regardless of internal sub-stepping.

A gradient planner makes one forward and one backward pass per Adam step, so per-step cost equals the number of model evaluations required to produce $\hat{z}_T$. Sequence-aware endpoint models ($\hat{\Phi}_\theta$ and G_γ^{seq}) produce $\hat{z}_T$ in one call (K_{inf} chained calls when Ours is scored at composition depth K_{inf}); scalar endpoint regressors (G_γ^{scalar}) chain K calls; rollout models (G_ψ) chain R calls. CEM evaluates $N_{\text{cand}} \cdot N_{\text{iter}}$ candidates per plan, each requiring as many forward calls as the model needs to score the trajectory. The Oracle is counted at its API granularity (`env.step`(chunk_h) once per chunk).

Table 5. Compute count per plan as a closed form in the planner and model parameters. Substituting $K = 8$, $R = 100$, $G = 50$, $N_{\text{cand}} N_{\text{iter}} = 512$ (and K_{inf} for the Ours rows) reproduces the $T = 2$ s column of Table 2; the G_ψ rows scale with $R = T/\Delta t$, the others are constant in T.

Model	Planner	Forward calls / plan	Value at T=2
$\hat{\Phi}_\theta$ (Ours)	gradient	$G\,K_{\text{inf}}$	$50\,K_{\text{inf}}$
G_γ^{scalar}	gradient	$K \cdot G$	400
G_ψ	gradient	$R \cdot G$	5,000
Oracle (true dynamics)	gradient	$K \cdot G$	400
Oracle (true dynamics)	CEM	$N_{\text{cand}} N_{\text{iter}} K$	4,096
G_ψ	CEM	$N_{\text{cand}} N_{\text{iter}} R$	51,200
G_ψ	MPPI	$N_{\text{cand}} N_{\text{iter}} R$	51,200

The closed-form structure makes the compute comparisons in Table 2 reducible to two ratios of model evaluations (taking Ours at single-call K_{inf}=1): (i) *one-call vs. chained-scalar* at the gradient planner ($\hat{\Phi}_\theta$ vs. G_γ^{scalar}): G vs. $K \cdot G$, a factor of $K = 8$; (ii) *gradient vs. CEM* at the same dynamics (Oracle gradient vs. Oracle CEM): KG vs. $N_{\text{cand}} N_{\text{iter}} K$, a factor of $N_{\text{cand}} N_{\text{iter}}/G = 512/50 \approx 10.2$. Composing these, $\hat{\Phi}_\theta$ vs. Oracle CEM is $\approx 81.9\times$ at K_{inf}=1, narrowing to $\approx 10.2\times$ at K_{inf}=8 where Ours matches the chunked baselines' call count (T-independent, since K is fixed). The first factor is the paper's contribution (the one-call sequence interface); the second is a known planner-paradigm effect that any gradient-friendly endpoint model would inherit.

Execution and aggregation. After planning, the chosen control sequence is executed on the true environment with zero-order hold over each control interval (per env step for the per-step plans). We record the terminal cost $C := \theta^2 + 0.1\,\dot{\theta}^2$ at $t = T$. The cost distribution over initial states is bimodal. Plans either reach the target or miss by a wide margin. We aggregate with the median rather than the mean, which would be dominated by the failed plans.

Replanning-depth Pareto sweep. Figure 2 traces, for each horizon T, how Ours's median terminal cost falls as the closed-loop receding-horizon replanning count $N_{\text{replan}} \in \{1, 2, 4, 8\}$ grows (the Ours rows of Table 7). More replanning helps monotonically at $T \geq 2$, the gain widening with T; the Pareto-optimal N_{replan} therefore grows from flat at T=1, where the horizon is already short, to depth-8 at $T \geq 4$, where it reaches near-Oracle medians (0.10 at T=4, 0.15 at T=5). One trained network underlies every point.

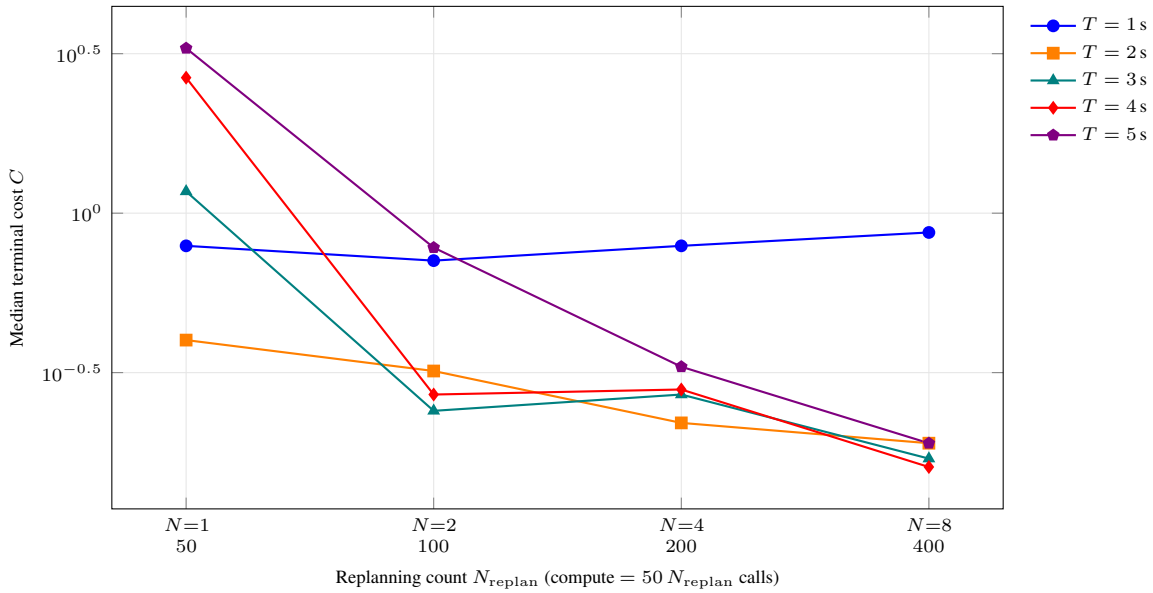

Figure 2. Closed-loop receding-horizon MPC replanning sweep for the any-horizon student $\hat{\Phi}_\theta$. Each curve is one planning horizon T, swept over replanning counts $N_{\mathrm{replan}} \in \{1, 2, 4, 8\}$ (compute = 50 N_{replan} forward calls per plan). Each leg replans the remaining $[t, T]$ toward the terminal goal, so more replanning lowers cost monotonically at $T \geq 2$, reaching near-Oracle medians at depth-8. The gain grows with T; at T=1 s the curve is flat, the horizon already too short for replanning to change the outcome. One trained network underlies every point.

B.4 Closed-loop MPC evaluation

We sweep the replanning count $N_{\mathrm{replan}} \in \{1, 2, 4, 8\}$ for the full Table 2 roster: Ours, the rollout model G_ψ (gradient, CEM, MPPI), the endpoint regressors $G_\gamma^{\mathrm{scalar}}$ and G_γ^{seq}, and the true-dynamics Oracle (gradient, CEM), each at K_{inf}=1 per leg. Open-loop (N_{replan}=1) executes the full T-step plan; receding-horizon MPC replans the remaining horizon $[t, T]$ toward the terminal goal at T from the realized state, executes the first T/N_{replan} of that plan, and repeats N_{replan} times. Every leg keeps the full terminal objective, so shortening legs never starves the goal. Same task as Table 2: 30 episodes, PYTHONHASHSEED=0; gradient planners use 50 Adam steps per leg, sampling planners 128 candidates over 4 iterations. Tables 6 and 7 are a pair from the *same* runs: the per-leg prediction MSE (predicted leg-endpoint vs. realized state) and planning quality. Compute per plan is the table's Compute column: G_ψ re-rolls every shrinking leg, so its cost grows with both T and N_{replan}; Ours alone is flat in T.

Under receding-horizon replanning every method improves monotonically with N_{replan}, with no short-T blow-up (Figure 3 shows the T=5 sweep). At T=1 replanning cannot help, the horizon is already short; at $T \geq 2$ deeper replanning helps monotonically, and depth-8 reaches near-Oracle medians at 100% reach (99 ± 2% at T=5 s across three training seeds). The decisive axis is compute: Ours is flat in T while G_ψ grows with it, so Ours matches Oracle-quality planning at 8× less compute and ties G_ψ gradient's 100% reach at 29× to 142× fewer forward calls; G_ψ's medians are marginally lower at that compute. The sampling planners (G_ψ and Oracle CEM, MPPI) reach low medians only at scattered long-T cells and a further order of magnitude in compute, and G_γ^{seq} never plans competitively. Per-leg prediction MSE (Table 6) falls in step with planning quality as the legs shorten: accurate per-leg prediction and adequate replanning improve together rather than trading off.

Task-trained controllers. The final block of Table 7 adds three controllers trained from scratch on the same `pendulum_full` transitions relabeled with the swing-up reward: model-based PETS (Chua et al., 2018) (a 5-model Gaussian ensemble with CEM-TS MPC) and the model-free IQL (Kostrikov et al., 2022) and CQL (Kumar et al., 2020) (γ=0.99, 10^5 gradient steps), evaluated on the same 30 init states and terminal cost. All three act at every env step, so their replan count is $R = T/\Delta t$, the natural group beyond the $N_{\mathrm{replan}} \in \{1, 2, 4, 8\}$ blocks above. PETS plans over a fixed 0.5s window each step; IQL and CQL are reactive policies with no lookahead. Because they are trained for this one reward, the cost that matters is training, not the per-plan forward calls of the zero-shot planners above. Against Ours's best per-horizon row (Table 2) the trained controllers lose both axes at short horizons ($T \leq 2$ s), and PETS and IQL never exceed 43% reach

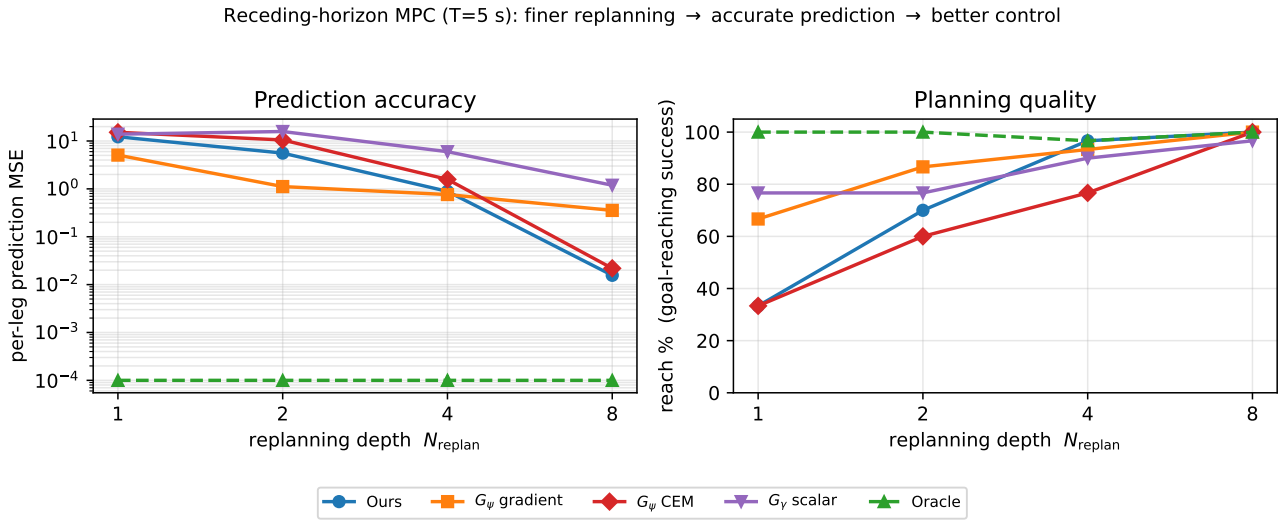

Figure 3. Receding-horizon MPC at the hardest horizon T=5 s, sweeping replanning depth $N_{\text{replan}} \in \{1, 2, 4, 8\}$ (one trained $\hat{\Phi}_\theta$, evaluated zero-shot). **Left:** per-leg prediction MSE (log scale); **right:** goal-reach rate. Shortening each open-loop leg drives the learned flow map's per-leg error down three orders of magnitude and lifts reach from 33% to 100%, matching the true-dynamics Oracle; the G_ψ planners plateau at higher prediction error. Same runs as Tables 6 and 7.

anywhere. CQL is the lone competitor: after 10^5 task-specific gradient steps it posts median 0.11 to 0.14 at $T \geq 3$ s, below Ours's open-loop best, but collapses at $T \leq 2$ s (median > 2). Closed-loop replanning closes the long-horizon gap: depth-8 Ours reaches 100% at every $T \geq 2$ at comparable median, zero-shot (Table 7).

B.5 Ablations

Smoothness control wins both absolute MSE and data efficiency. The smoothness-controlled student has the lowest in-distribution MSE at both 100% and 20% data, and the lowest scarcity ratio. Independent-uniform synthetic ties absolute MSE at 100% but degrades 1.51× under scarcity.

Sequence input matters at planning time. A scalar-input variant of the law-first student is distilled from f_ϕ but takes one action per call. Its in-distribution MSE sits at $4.92/1.08$, close to the independent-uniform sequence variant's $4.81/1.35$; gradient planning costs $R\times$ more forward calls per Adam step than the one-call sequence student, losing the compute advantage of Section 5.1.

B.6 Scarcity-recipe ablation

Velocity matching alone preserves a sharp typical-trajectory error (low median) but admits a heavy off-manifold tail at sparse data. Endpoint loss at large supervision horizon diverges from coarse-substep integration; at short horizon it destroys the median. The integrated trajectory loss is stable across horizons and monotone in H on the mean: longer supervision cuts the tail. H=150 hits a stability ceiling.

The integrated loss assumes zero-order hold (ZOH) over the H-step rollout window: the action is sampled once at t_0 and reused for all H chained one-step predictions. The training data is generated with action-hold windows of 300 steps ($\gg H$), so every sample window we draw is constant-action by construction. The supervision target z_{t_0+h} matches the integration under the same constant action that drove the data; the loss has no action-mismatch bias.

TFH (Trajectory-Force, dense parallel per-step supervision evaluated at observed (z, u) tuples) tests whether on-manifold supervision can replace the chained H=100 loss. TFH alone has the lowest median (0.081) but the mean stays at 7.41, above the recipe's 4.86. vm+TFH sharpens median (0.096) but the mean climbs to 13.01. Adding chained-H=20 to TFH makes things worse on both axes. Only long-chained supervision (H=100) reaches a low mean. TFH and chained supervision target different failure modes: TFH supervises at observed (z, u) tuples and sharpens the typical case; chained supervision visits off-manifold states the model drifts into during training and catches the mean tail. The scarcity heavy-tail problem is off-manifold; only chained supervision resolves it.

For varying-action datasets, the cen-5 velocity-matching term can be replaced by the integrated trajectory loss alone. At full data this swap is neutral or better: integrated-alone $H{=}100$ reaches teacher rollout mean 3.76 on the constant-action probe, matching vm-only's 2.36 and vm+integrated-$H{=}100$'s 4.70. Under scarcity the cen-5 anchor adds a $1.6\times$ polish on the mean (integrated-alone $H{=}100$ at 10%: 7.74 vs vm+integrated-$H{=}100$'s 4.86), but the chained integrated loss carries the structural lift relative to vm-only's 17.97. The recipe choice is therefore between cen-5 + integrated (when the training data has constant-action segments long enough for the stencil) and integrated alone (for arbitrary-action data).

Scarcity and the synthetic budget. The per-batch teacher signal is independent of the real-data count: the store-all-intermediates compilation draws fresh $(z, u_{[t,t+T]})$ pairs and integrates the frozen f_ϕ to produce endpoints, so the student sees the same synthetic-distribution coverage no matter how many real trajectories seeded it. What scarcity removes is real-data coverage of the state space: the Scarce (10%) student in Tables 3 and 4 keeps short-horizon planning but degrades from $T{=}3$ s and fails the longest horizon ($T{=}5$ s), where the missing trajectories would have anchored f_ϕ. The synthetic compilation is a one-time training cost, paid against the constant 50-call inference budget at deployment.

B.7 Horizon sampling: continuous vs. anchored

Ours samples its supervision horizon continuous-uniformly over $T \in [0.08, 5.12]$ s with semigroup weight 0. The anchored ablation samples the horizon at nine discrete anchors instead and turns the semigroup loss on at weight 0.03; the store-all-intermediates action distribution, architecture, optimizer, budget, and teacher are identical. The semigroup-weight rule follows the sampling density: under sparse anchor sampling the loss is needed to enforce consistency at the horizons the anchors miss (dropping it doubles MSE), while under dense continuous-T sampling the data already supplies the consistency and the best weight is 0.

Ours stays accurate across the whole range. The anchored ablation fails outright off its anchors: its training never visits horizons between them, so only anchor-aligned queries are in-support. At its $T{=}5.12$ s anchor it recovers, yet still trails Ours with wider seed-variance (Table 3). A horizon grid is a brittle interface; continuous coverage removes the failure mode and is more accurate.

B.8 Student recipe ablation

We ablate the student synthetic-action distribution on per-step varying-action MSE at $T = 5.12$ s ($n = 256$). Each row is a separate training run; same teacher (vm-only at 100% data) and 48k-step budget. The K-chunked-anchor variant samples K discrete control chunks per sequence (constant action per chunk); R-step variants sample R per-step actions per sequence with segment count B controlling smoothness.

K-chunked anchor training fails by $\approx 50\times$ on per-step varying-action input. R-step variants fix it. Uniform-B freeA-extB has the lowest MSE but loses planning at open-loop short-T ($T{=}2$ $N_{\text{replan}}{=}1$ median 4.27 vs 0.59 for log-uniform B on the same student): uniform-B overweights high-frequency segments, leaving the smooth-piecewise regime where planning solutions live underrepresented. Log-uniform B gives each smoothness octave equal weight ($\sim$37% of samples at $B \leq 16$ vs $\sim$6% under uniform-B), accurate at both extremes. All rows use the anchored horizon schedule; the adopted action distribution carries over unchanged to the continuous-horizon Ours (Section B.7).

B.9 Generalization to unseen horizons

We test whether the two-stage construction transfers beyond the training horizon range. The first two rows hard-cap training data at $h \leq 50$ steps; the third keeps the same f_ϕ but trains the student on synthetic teacher rollouts up to $h \leq 256$, well beyond the raw-data limit. Test sequences have action varying at every env step.

The $h \leq 50$ students fail to generalize past their training cap: G_γ's MSE rises from 0.59 at $h{=}50$ to 18.27 at $h{=}100$; the $h \leq 50$ Ours student degrades to 62.98. The $h \leq 256$ Ours student keeps MSE under 2.1 across all three horizons, $9\times$ lower than G_γ and $31\times$ lower than the $h \leq 50$ Ours student at $h{=}100$. Horizon extrapolation requires using f_ϕ to manufacture supervision past the raw-data range.

Table 6. Per-leg prediction MSE during the same MPC runs as Table 7, grouped by N_{replan}. Cells: mean / median over legs and episodes. Bold marks the lowest non-oracle median in each N_{replan} block at each T. Per-leg prediction error falls as N_{replan} shortens each leg, in step with the planning-quality gains of Table 7; the Oracle uses true dynamics, so its per-leg prediction error is zero by construction.

N_{replan}	Method	T=1	T=2	T=3	T=4	T=5
1	Ours $\hat{\Phi}_\theta$	0.22/0.019	0.96/**0.29**	5.58/3.08	10.3/8.39	12.3/12.3
	G_ψ grad	0.19/0.056	2.84/0.61	6.59/**0.83**	2.94/**1.05**	5.08/**2.13**
	G_ψ CEM	0.037/0.015	9.25/2.84	23.6/18.8	20.8/11.6	15.2/11.6
	G_ψ MPPI	0.023/**0.009**	9.94/3.86	14.0/8.41	12.3/7.86	15.0/10.9
	G_γ^{scalar}	3.49/1.56	17.6/11.3	21.3/13.7	15.7/12.9	13.9/8.78
	G_γ^{seq}	12.0/8.79	9.34/5.42	14.1/8.75	18.0/13.9	17.8/15.8
	Oracle grad	0.000/0.000	0.000/0.000	0.000/0.000	0.000/0.000	0.000/0.000
	Oracle CEM	0.000/0.000	0.000/0.000	0.000/0.000	0.000/0.000	0.000/0.000
2	Ours $\hat{\Phi}_\theta$	0.015/0.009	0.058/**0.025**	1.33/**0.080**	2.59/**0.18**	5.59/2.09
	G_ψ grad	0.072/0.061	0.35/0.22	1.32/0.18	0.98/0.36	1.12/**0.14**
	G_ψ CEM	0.032/0.012	0.24/0.17	4.54/2.96	15.0/13.5	10.5/8.43
	G_ψ MPPI	0.023/**0.006**	0.18/0.089	4.48/2.02	15.7/14.1	11.1/9.27
	G_γ^{scalar}	1.83/0.64	3.50/1.44	9.44/5.12	11.8/7.70	15.8/12.7
	G_γ^{seq}	13.0/13.3	13.1/11.4	16.0/14.2	8.69/5.77	12.8/12.2
	Oracle grad	0.000/0.000	0.000/0.000	0.000/0.000	0.000/0.000	0.000/0.000
	Oracle CEM	0.000/0.000	0.000/0.000	0.000/0.000	0.000/0.000	0.000/0.000
4	Ours $\hat{\Phi}_\theta$	0.005/**0.003**	0.009/**0.004**	0.54/0.035	0.58/**0.035**	0.89/0.077
	G_ψ grad	0.024/0.012	0.021/0.014	0.032/**0.019**	0.49/0.097	0.76/**0.046**
	G_ψ CEM	0.008/**0.003**	0.017/0.013	0.069/0.057	0.26/0.16	1.58/0.65
	G_ψ MPPI	0.006/**0.003**	0.020/0.012	0.055/0.037	0.29/0.19	1.65/1.12
	G_γ^{scalar}	0.41/0.18	0.60/0.47	2.36/0.63	2.46/1.66	5.98/3.57
	G_γ^{seq}	6.02/6.37	14.0/13.9	21.1/21.1	17.6/16.1	16.9/16.2
	Oracle grad	0.000/0.000	0.000/0.000	0.000/0.000	0.000/0.000	0.000/0.000
	Oracle CEM	0.000/0.000	0.000/0.000	0.000/0.000	0.000/0.000	0.000/0.000
8	Ours $\hat{\Phi}_\theta$	0.086/0.002	0.002/**0.002**	0.25/**0.002**	0.088/**0.003**	0.016/**0.011**
	G_ψ grad	0.009/0.006	0.092/0.011	0.015/0.015	0.17/0.011	0.35/0.028
	G_ψ CEM	0.004/**0.001**	0.011/0.008	0.008/0.005	0.096/0.009	0.022/0.017
	G_ψ MPPI	0.002/**0.001**	0.009/0.006	0.094/0.008	0.094/0.009	0.023/0.019
	G_γ^{scalar}	0.15/0.065	0.31/0.15	0.45/0.42	0.82/0.31	1.19/0.54
	G_γ^{seq}	2.72/2.98	6.19/6.43	10.3/10.7	14.8/15.0	17.7/18.4
	Oracle grad	0.000/0.000	0.000/0.000	0.000/0.000	0.000/0.000	0.000/0.000
	Oracle CEM	0.000/0.000	0.000/0.000	0.000/0.000	0.000/0.000	0.000/0.000

Table 7. Closed-loop MPC planning quality, grouped by replanning count N_{replan}; matched N_{replan} is not matched compute (Compute column: total forward calls per plan). Cells: median terminal cost / reach %. Bold marks the best non-oracle median and reach in each N_{replan} block at each T. The final block holds task-trained controllers that act every env step, so their replan count is $R{=}T/\Delta t$: PETS plans over a fixed 0.5s window each step, while IQL and CQL are reactive policies with no lookahead. For these the relevant cost is training, so Compute reports 10^5 gradient steps ("tr.") except PETS's $\approx 10^7$ inference calls; bold marks the best within the block.

N_{replan}	Method	Compute	$T{=}1$	$T{=}2$	$T{=}3$	$T{=}4$	$T{=}5$
1	Ours $\hat{\Phi}_\theta$	50	**0.79/83%**	**0.40/97%**	1.17/**73%**	2.66/40%	3.29/33%
	G_ψ grad	$\approx$2.5kT	0.88/80%	0.52/77%	**0.50/73%**	**0.49**/73%	0.87/67%
	G_ψ CEM	$\approx$25kT	1.70/57%	1.33/60%	5.13/20%	3.39/33%	3.87/33%
	G_ψ MPPI	$\approx$25kT	2.29/43%	1.89/53%	3.31/30%	4.22/33%	3.59/37%
	G_γ^{scalar}	400	1.33/70%	0.92/83%	1.30/63%	0.63/**87%**	**0.69/77%**
	G_γ^{seq}	50	4.06/3%	7.49/10%	4.86/30%	6.49/17%	6.29/17%
	Oracle grad	400	0.82/80%	0.42/97%	0.18/100%	0.095/100%	0.22/100%
	Oracle CEM	4,096	1.86/60%	0.71/73%	3.69/30%	0.76/80%	2.15/47%
2	Ours $\hat{\Phi}_\theta$	100	**0.71/83%**	0.32/**100%**	**0.24/100%**	**0.27/90%**	0.78/70%
	G_ψ grad	$\approx$3.6kT	0.72/**83%**	0.47/**100%**	0.42/87%	0.53/**90%**	**0.70/87%**
	G_ψ CEM	$\approx$37kT	1.75/63%	**0.26**/87%	1.54/53%	6.81/17%	1.04/60%
	G_ψ MPPI	$\approx$37kT	1.92/57%	**0.26**/83%	1.37/57%	6.75/13%	1.45/60%
	G_γ^{scalar}	800	0.83/80%	0.85/87%	1.03/77%	1.30/77%	1.28/77%
	G_γ^{seq}	100	4.58/0%	4.23/20%	6.98/13%	4.42/23%	5.42/17%
	Oracle grad	800	0.67/83%	0.35/100%	0.31/100%	0.49/90%	0.40/100%
	Oracle CEM	8,192	1.87/60%	0.028/100%	0.16/97%	1.35/57%	5.97/30%
4	Ours $\hat{\Phi}_\theta$	200	0.79/**83%**	0.22/**100%**	0.27/**100%**	0.28/97%	**0.33/97%**
	G_ψ grad	$\approx$6.4kT	**0.72**/80%	0.064/**100%**	0.30/97%	0.53/**100%**	0.48/93%
	G_ψ CEM	$\approx$66kT	1.92/60%	0.007/97%	0.077/90%	0.74/70%	0.69/77%
	G_ψ MPPI	$\approx$66kT	1.87/57%	**0.005**/93%	**0.023**/87%	**0.21**/77%	0.76/60%
	G_γ^{scalar}	1,600	0.76/77%	0.33/93%	0.55/77%	0.69/80%	0.58/90%
	G_γ^{seq}	200	3.85/17%	5.08/10%	5.78/10%	5.20/23%	4.21/20%
	Oracle grad	1,600	0.65/80%	0.20/100%	0.23/100%	0.30/97%	0.32/97%
	Oracle CEM	16,384	1.80/50%	0.005/97%	0.001/100%	0.011/100%	0.007/97%
8	Ours $\hat{\Phi}_\theta$	400	0.87/80%	0.19/**100%**	0.17/**100%**	0.16/**100%**	0.19/**100%**
	G_ψ grad	$\approx$11.6kT	**0.66/83%**	0.13/**100%**	0.12/**100%**	0.045/**100%**	0.14/**100%**
	G_ψ CEM	$\approx$119kT	1.80/63%	**0.005/100%**	**0.003/100%**	**0.002/100%**	**0.004/100%**
	G_ψ MPPI	$\approx$119kT	1.58/63%	**0.005**/97%	0.004/**100%**	**0.002/100%**	0.005/**100%**
	G_γ^{scalar}	3,200	0.73/77%	0.33/93%	0.57/**100%**	0.37/**100%**	0.28/97%
	G_γ^{seq}	400	3.74/23%	7.46/13%	5.59/17%	7.63/13%	6.12/13%
	Oracle grad	3,200	0.57/83%	0.17/100%	0.11/100%	0.13/100%	0.13/100%
	Oracle CEM	32,768	1.39/57%	0.004/100%	0.000/100%	0.000/100%	0.001/100%
R	PETS	$\approx 10^7$	3.49/17%	4.53/13%	4.11/27%	2.71/43%	2.98/37%
	IQL (reactive)	10^5 tr.	3.51/13%	4.66/7%	5.60/7%	6.14/10%	5.33/13%
	CQL (reactive)	10^5 tr.	**2.27/37%**	**2.12/47%**	**0.14/90%**	**0.12/97%**	**0.11/97%**

Table 8. Endpoint MSE at $T{=}2$ s in-distribution on $n = 256$ test sequences with action varying at every env step. Cells: mean / median. Scarcity ratio = 20% mean / 100% mean.

Variant	100% MSE	20% MSE	Scarcity ratio
Scalar distilled (no sequence input)	4.92/1.08	7.77/2.31	1.58
Sequence, independent-uniform synthetic	4.81/1.35	7.26/3.03	1.51
Sequence, smoothness-controlled (ours)	**3.86/0.44**	**3.92/0.73**	**1.02**

Table 9. Teacher step-by-step rollout MSE at $T = 5.12$ s on $n = 256$ test sequences with action varying at every env step (Table 3 probe), 10% training data. Mean and median are reported separately to expose the heavy-tail asymmetry.

Teacher loss	Mean	Median
G_ψ-10% (rollout baseline)	9.11	0.90
vm-only (cen-5)	17.97	0.210
cen-3 vm-only	12.58	0.222
vm + endpoint loss (H=50)	7.61	0.433
vm + integrated (H=10)	12.89	0.112
vm + integrated (H=50)	8.84	0.579
vm + integrated (H=100, recipe)	**4.86**	0.117
vm + integrated (H=150)	6.85	0.445
TFH=100 (no vm)	7.41	**0.081**
TFH=100 + chained-H=20 (no vm)	14.79	6.82
vm + TFH=100	13.01	0.096
vm + TFH=100 + chained-H=20	17.48	0.218
vm + TFH=100 + chained-H=100	6.44	0.785
integrated alone (H=1, no vm)	23.30	0.140
integrated alone (H=100, no vm)	7.74	1.56

Table 10. Endpoint MSE (mean / median) across arbitrary horizons T on $n = 256$ per-step varying-action test sequences, single call. The probed T lie off the anchored ablation's nine-anchor training set, so the anchored student is queried outside its horizon support; Ours is trained at continuous horizons and treats every T as in-distribution.

T (s)	0.5	1	1.5	2	2.5	3	5
Ours mean	0.24	0.13	0.98	1.73	2.36	2.74	4.94
Ours med	0.001	0.009	0.018	0.034	0.043	0.107	0.699
Anchored mean	1.02	6.62	32.94	68.22	81.27	97.69	306.17
Anchored med	0.133	4.176	21.40	31.45	35.28	40.97	145.16

Table 11. Student recipe ablation: per-step varying-action MSE at $T = 5.12$ s.

Student recipe	MSE mean	MSE median
K-chunked anchors ($K \in \{8, 16, 32, 64\}$)	324.4	151.5
R-step bucket ($R \in \{8, \ldots, 256\}$, B=R)	7.69	3.03
R-step freeA-extB (uniform $B \in [1, R]$)	4.76	0.63
R-step logB, allint (adopted)	**6.16**	**1.82**

Table 12. Endpoint MSE (mean) at in- and out-of-distribution horizons on $n = 256$ test sequences with action varying at every env step. Training horizons are capped at $h = 50$. Entries marked * are unseen at training time.

Endpoint MSE	h=50	h=75*	h=100*
G_γ, $h \le 50$ (raw-data limit)	0.59	11.82	18.27
Ours $\hat{\Phi}_\theta$, $h \le 50$ synthetic	2.40	23.43	62.98
Ours $\hat{\Phi}_\theta$, $h \le 256$ synthetic	**0.45**	**1.35**	**2.04**